

Appendix C: IEEE 830 Template

Software Requirements Specification

for

Hotel Reservation System

Version 1.0

Prepared by Nicolás Pérez Gómez

Lodz University of Technology - IFE

01-04-2026

Table of Contents

«Table of Contents»

Revision History

Name	Date	Reason for Changes	Version
Nicolas Perez Gomez	02-04-2026	Initial Version	1.0

«Page break»

1. Introduction

1.1. Purpose

The purpose of this document is to specify the software requirements and design for the Hotel Reservation System (HRS), Version 1.0. This Software Requirements Specification (SRS) covers the entire scope of the product, detailing the functionalities required for guests to search for and book rooms online, as well as for hotel staff to manage reservations, room availability, check-ins, check-outs, and customer billing. This document serves as a binding contract and reference guide between the development team and the hotel management stakeholders.

1.2. Document Conventions

This document follows the IEEE 830 standard for Software Requirements Specifications. To ensure clarity, traceability, and adherence to the SMART criteria outlined in the project guidelines, the following conventions are applied throughout the document:

Requirement Identifiers: Functional requirements are uniquely identified with the prefix "FR-" (e.g., FR-01, FR-02), and Non-Functional requirements are identified with the prefix "NFR-" (e.g., NFR-01).

Terminology of Obligation: The word "shall" is used to define mandatory requirements that the system must strictly fulfill. The word "should" indicates a highly desirable feature, and "may" implies an optional feature.

Prioritization: Every individual requirement statement has its own specific priority level assigned to it (e.g., High, Medium, Low). Priorities are not assumed to be inherited from higher-level feature descriptions.

Topographical Emphasis: Important system entities, specifically the system Actors (e.g., Guest, Receptionist, Admin) and core UML Classes, are written in bold text the first time they are introduced or when emphasizing their specific interactions with the system.

1.3. Intended Audience and Reading Suggestions

This Software Requirements Specification (SRS) is a technical and managerial document intended for a diverse group of stakeholders.

- The primary audiences include:

Hotel Management and Stakeholders: The clients who requested the system and need to ensure it meets their business needs.

Project Managers: The individuals responsible for tracking project progress, scope, and resource allocation.

Development Team (Software Engineers and UI/UX Designers): The technical staff responsible for designing, writing, and implementing the code.

Quality Assurance (QA) Testers: The personnel responsible for verifying that the software meets all defined requirements without defects.

- **Document Organization:**

The remainder of this document is organized as follows:

Section 2 (Overall Description): Provides a high-level overview of the system, including user classes, operating environments, and design constraints.

Section 3 (Specific Requirements): Details the precise Functional Requirements (FR) and Non-Functional Requirements (NFR) that the development team must build.

Appendices: Contains the structural and behavioral UML models (Use Case, Class, Sequence, State, and Activity diagrams) designed during the analysis phase.

- **Suggested Reading Sequence:**

Hotel Management and Stakeholders should begin with Section 1 (Introduction) and Section 2 (Overall Description) to verify the product scope and general characteristics. They may skim Section 3.

Project Managers should read the entire document sequentially to fully grasp both the business goals and the technical scope.

Developers should briefly review Sections 1 and 2 for context, but must focus deeply on Section 3 (Specific Requirements) and the UML diagrams in the Appendices to guide their architectural and coding decisions.

QA Testers should concentrate heavily on Section 3, using the specific and measurable FRs and NFRs to write their test cases, while referencing Section 2 for environment constraints.

1.4. Product Scope

Currently, many small and medium-sized hotels manage their reservations using outdated, fragmented tools such as physical ledgers or basic spreadsheets. This manual approach often leads to critical business issues: double-booked rooms (overbooking), lost customer data, inefficient communication between staff shifts, and painfully slow check-in/check-out processes for guests. Furthermore, without a direct online booking platform, hotels lose potential revenue to third-party booking agencies that charge high commission fees.

The Hotel Reservation System (HRS) is a centralized, web-based software solution designed to resolve these inefficiencies by automating the entire reservation lifecycle. The system will allow prospective guests to view real-time room availability and securely book their stays online. Simultaneously, it will provide hotel staff with a unified dashboard to manage check-ins, check-outs, room statuses, and customer billing.

- **Benefits, Objectives, and Goals:**

Eliminate Overbooking: By maintaining a single, real-time database of room availability, the system guarantees that a room cannot be booked by two different parties for the same dates.

Enhance Customer Experience: Provide a seamless, self-service online booking portal and reduce physical waiting times at the reception desk by at least 50%.

Increase Direct Revenue: Align with the corporate strategy of increasing direct sales by providing a reliable booking engine on the hotel's own website, bypassing third-party commissions.

Improve Administrative Efficiency: Automate invoice generation and allow management to easily track occupancy rates and financial reports.

1.5. References

«List any other documents or Web addresses to which this document refers or was based on. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location. Include the assignment document in the references.»

2. Overall Description

2.1. Product Perspective

The Hotel Reservation System (HRS) is a new, self-contained software product. It is being developed as a direct replacement for the legacy manual systems (such as physical logbooks and disjointed Excel spreadsheets) currently used by the hotel staff.

While it operates as an independent web application, it is not entirely isolated. The system will need to interact with a few external services to function at full capacity:

Payment Gateway Interface: The system will integrate with a secure third-party payment processor (e.g., Stripe or PayPal) to handle online credit card transactions for room deposits.

Email Service Provider: The system will connect to an external SMTP server to automatically dispatch booking confirmations, invoices, and cancellation notices to the guests' email addresses.

Database System: It will rely on a centralized relational database (e.g., MySQL or PostgreSQL) to store persistent data regarding users, rooms, and historical reservations safely.

2.2. Product Features

The system provides the following main features based on user roles:

Guest Module: Searching for rooms, filtering search results, and booking rooms.

Payment Module: Processing payments and calculating totals.

Notification Module: Sending booking confirmations via an external email system.

Reception Module: Staff login and management of guest check-ins and check-outs.

Admin Module: Generating system reports and managing staff.

2.3. User Classes and Characteristics

The system interacts with the following primary user classes: Guest: The end-user who interacts with the system remotely to search for rooms and make reservations. The system will record their name, email, phone, and assign them a guest ID upon booking. Receptionist: Hotel staff responsible for daily operations. They require authentication (login) to access the system and manage check-in and check-out functions. Admin: The user with the highest system privileges. They require authentication to generate hotel reports and manage staff members.

2.4. Operating Environment

The Hotel Reservation System (HRS) is designed as a centralized, web-based application, meaning it will operate across various client-side and server-side environments without requiring localized installation for the users.

Client-Side Environment:

- **Hardware:** Any standard desktop, laptop, tablet, or smartphone with internet connectivity.
- **Operating Systems:** OS agnostic (Windows, macOS, Linux, Android, iOS).
- **Web Browsers:** The system must be fully functional and responsive on modern web browsers (e.g., Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge).

Server-Side Environment:

- **Hardware Platform:** Cloud-based hosting infrastructure or standard dedicated web servers capable of handling simultaneous web requests.
- **Database:** A centralized relational database management system, such as MySQL or PostgreSQL, is required for secure and persistent data storage regarding users, rooms, and historical reservations.

Coexisting Software & Integrations:

- The system must securely communicate and coexist with third-party Payment Gateways (e.g., Stripe, PayPal) to handle online credit card transactions.
- The system requires a reliable connection to an external Email Service Provider (SMTP server) to automatically dispatch booking confirmations and invoices.

2.5. Design and Implementation Constraints

The development and implementation of the Hotel Reservation System (HRS) are subject to the following constraints:

Regulatory and Compliance Constraints:

- **Data Privacy:** Because the system collects and stores sensitive user information (such as guest names, emails, and phone numbers), it must comply with regional data protection regulations (such as GDPR) regarding the secure storage and handling of personally identifiable information (PII).
- **Payment Security:** The system will handle online credit card transactions for room deposits. Therefore, its integration with the Payment Gateway must strictly adhere to PCI-DSS (Payment Card Industry Data Security Standard) compliance rules, meaning no raw credit card data should be stored directly in the hotel's local database.

Technological Constraints:

- **Database Systems:** The development team is restricted to using a centralized relational database system, specifically MySQL or PostgreSQL, to ensure data integrity and safe persistent storage.
- **Architecture:** The product must be built as a centralized, web-based software solution. This restricts the use of desktop-native frameworks and requires the UI to be responsive and accessible via web browsers.

Interface and Integration Constraints:

- **Third-Party Dependency:** The system cannot process payments or send emails natively; it is entirely dependent on external service providers for these core functions. It must integrate with a secure third-party payment processor (e.g., Stripe or PayPal) and an external SMTP server for email dispatch. The software must be designed to gracefully handle connection timeouts or API changes from these external services.

Security Considerations:

- All web traffic must be encrypted using standard secure communication protocols (HTTPS/SSL) to protect guest data during the booking process and to secure receptionist and admin login credentials.

2.6. User Documentation

To ensure smooth operation and adoption of the Hotel Reservation System (HRS), the following user documentation components will be delivered along with the software:

- Guest Online Help & FAQ (Web-based): Given that the guest interface is intended for the general public, it is designed to be highly intuitive. Formal manuals will not be required for Guests. Instead, the system will include embedded contextual tooltips during the booking and payment process, as well as a dedicated FAQ web page (HTML format) covering topics like search filters, reservation modifications, and payment methods.
- Receptionist User Manual (PDF and Digital Wiki): A comprehensive operational guide will be provided for the hotel staff. This document will detail the step-by-step procedures for staff authentication (login), managing guest arrivals (manageCheckIn), processing departures (manageCheckOut), and handling daily front-desk operations. It will be delivered as a downloadable PDF and as an accessible internal wiki page within the staff module.
- System Administrator Guide (PDF): A specialized technical and operational manual exclusively for the Admin user. This guide will cover the highest-level privileges, including how to generate system-wide operational reports (generateReports) and how to properly add, edit, or remove employee credentials (manageStaff).
- Staff Training Tutorials (Video Format): Short, step-by-step video tutorials demonstrating the core workflows for the Receptionist and Admin (e.g., "How to process a Check-in", "How to generate a monthly report"). These will be delivered in standard MP4 format and hosted on the internal hotel staff portal.

2.7. Assumptions and Dependencies

The development and successful operation of the Hotel Reservation System (HRS) are based on the following assumptions and dependencies:

Assumptions:

- Continuous Internet Connectivity: It is assumed that the hotel premises will maintain a stable, high-speed internet connection. Since the system is web-based, the Receptionist and Admin require continuous online access to manage check-ins, check-outs, and reports.
- Hardware Availability: It is assumed that the hotel management will provide the necessary hardware (e.g., front-desk computers, laptops, or tablets) equipped with modern web browsers for staff members to access the system.
- User Literacy: It is assumed that both the hotel staff and the guests possess basic digital literacy and are familiar with standard web navigation and online forms.
- Open Source/Commercial Database: It is assumed that standard relational database management systems (like MySQL or PostgreSQL) will be utilized and that no proprietary, legacy database systems need to be integrated.

Dependencies:

- Payment Gateway API: The system is strictly dependent on the continuous availability and API stability of the third-party Payment Gateway (e.g., Stripe, PayPal). Any downtime or deprecation of the payment provider's API will directly halt the system's ability to process paid reservations.

- **Email Service Provider:** The Notification Module depends entirely on a third-party SMTP server or Email API (e.g., SendGrid, Mailchimp) to dispatch booking confirmations. If this external service fails, guests will not receive their automated confirmation emails.
- **Cloud/Server Hosting Uptime:** The system's overall availability and performance are dependent on the Service Level Agreement (SLA) of the chosen cloud infrastructure or web hosting provider where the application and database will be deployed.

3. System Features

The Hotel Reservation System (HRS) is composed of several key functional modules designed to serve different user roles and external system interactions. The following is a brief overview of these core features, which are detailed further in Section 5.

3.1. Guest Reservation Operations

This feature acts as the primary interface for potential customers. It allows guests to search the hotel's inventory for available rooms, apply specific search filters, and secure a booking for their desired check-in and check-out dates.

3.2. Payment and Notification Processing

This feature handles the backend operations that occur immediately after a guest requests a booking. It integrates with an external Payment Gateway to securely process credit card transactions and subsequently triggers an external Email System to dispatch formal booking confirmations to the guest.

3.3. Reception Operations

This feature provides the necessary operational tools for the hotel's front desk staff. It includes secure login capabilities and enables receptionists to effectively manage the daily flow of guest arrivals (check-in) and departures (check-out).

3.4. Administration Operations

This feature encompasses the managerial tools reserved for the Admin user. It grants the highest level of system access, allowing the administrator to generate comprehensive hotel operational reports and securely manage the credentials of the hotel staff.

4. External Interface Requirements

4.1. User Interfaces Overview

The Hotel Reservation System (HRS) will feature a web-based Graphical User Interface (GUI) designed to be intuitive, responsive, and accessible across different devices. The system provides distinct interfaces based on user roles:

High-Level Functionality by Screen:

- **Guest Landing & Search Interface:** A public-facing web page where Guests can input check-in/check-out dates and guest numbers. Users interact via text fields, dropdowns, and a "Search" button.
- **Search Results & Filtering Interface:** Displays a list of available rooms fetched from the database. Guests can interact with checkboxes or sliders to filter by room type or price. Each room card will have a "Book Now" button.
- **Booking & Payment Form:** A secure page where the Guest enters personal details (Name, Email, Phone) and credit card information. The system will display a loading animation during payment processing and return a clear success or error message based on the Payment Gateway's response.

- **Staff Login Screen:** A standardized authentication screen requiring a Username/Employee ID and Password, utilized by both Receptionists and Admins.
- **Receptionist Dashboard:** A tabular GUI displaying current-day arrivals and departures. It includes action buttons for `manageCheckIn()` and `manageCheckOut()`, allowing the Receptionist to update room occupancy statuses seamlessly.
- **Admin Dashboard:** An expanded dashboard containing data visualization tools (charts/tables) for `generateReports()`, and a management module to add, edit, or remove staff credentials (`manageStaff()`).
- **Interface Minimum Requirements & Standards:**
- **Responsive Design:** The Guest interface must be fully mobile-responsive (adjusting to smartphones, tablets, and desktops). The Staff/Admin interfaces are optimized for desktop and tablet resolutions.
- **Navigation:** A persistent top navigation bar must appear on all screens, containing a "Home" button, "Contact/Help" links, and a "Logout" button for authenticated staff.
- **Feedback & Error Handling:** All forms must include real-time input validation (e.g., highlighting invalid email formats in red). Critical actions (like canceling a reservation) must prompt a confirmation modal dialog. System errors must be displayed in standardized alert banners at the top of the screen.

4.2. Hardware Interfaces

As a centralized, cloud-hosted web application, the HRS does not require direct, low-level integration with custom or proprietary physical hardware (such as physical keycard encoders, unless specified as a future extension). The hardware interfaces are defined by standard networking and client-server interactions:

- **Client Devices:** The software must interface with standard input/output hardware of modern computing devices (PCs, laptops, tablets, smartphones) including keyboards, touchscreens, and mice. This is abstracted through the user's web browser.
- **Server Hardware:** The application will reside on standard cloud server hardware. The logical interface between the client hardware and server hardware will be facilitated over standard network interfaces using TCP/IP protocols.
- **Communication Protocols:** All data exchanged between the client hardware (browser) and the hosting server must be transmitted over HTTPS (port 443) to ensure encrypted data payloads, especially concerning user authentication and booking information.

4.3. Software Interfaces

5. System Features/Modules

5.1. Guest Reservation Operations

5.1.1 Description and Priority

This feature encompasses all functionalities required for a Guest to search the hotel inventory, filter results, and initiate a room booking. Priority: High.

5.1.2 Stimulus/Response Sequences

Condition: The Guest submits search criteria (check-in/check-out dates).

Response: The system displays a list of available rooms matching the dates.

Condition: The Guest applies filters (e.g., room type, price).

Response: The system narrows down the displayed search results accordingly.

Condition: The Guest selects a room and clicks "Book Room".

Response: The system initiates the makeReservation() process and requests guest details.

5.1.3 Functional Requirements

- REQ-1.1: Upon receiving search criteria, the system shall query getAvailableRooms() and display all unoccupied rooms for the selected dates.
- REQ-1.2: Upon the user applying search filters, the system shall instantly filter the displayed rooms based on roomType and price.
- REQ-1.3: Upon selecting a room to book, the system shall execute makeReservation() and instantiate a new Reservation record with the checkInDate, checkOutDate, and a generated reservationID.
- REQ-1.4: The system shall calculate and display the final cost utilizing the calculateTotal() function based on the selected room's price.
- REQ-1.5: The system shall display an anticipated error message if the Guest attempts to book a room that has just been reserved by another user.

5.2. Payment and Notification Processing

5.2.1 Description and Priority

This feature manages the background operations of processing financial transactions securely via a Payment Gateway and dispatching automated emails. Priority: High.

5.2.2 Stimulus/Response Sequences

Condition: The Guest submits their payment details to finalize the booking.

Response: The system transmits the data to the Payment Gateway and awaits verification.

Condition: The Payment Gateway returns a "Success" status.

Response: The system finalizes the booking and sends a confirmation email.

Condition: The Payment Gateway returns a "Declined" status.

Response: The system aborts the reservation and alerts the Guest.

5.2.3 Functional Requirements

- REQ-2.1: Upon submission of payment details, the system shall invoke processPayment() to securely send the amount and method to the external Payment Gateway.
- REQ-2.2: Upon receiving a successful status from the Payment Gateway, the system shall generate a paymentID and change the Reservation status to "Confirmed".
- REQ-2.3: Upon confirming the reservation, the system shall invoke the secondary Email System to send a booking confirmation to the Guest's email.
- REQ-2.4: Upon successful booking, the system shall execute updateStatus(status: true) to mark the room's isOccupied attribute as true for the specified dates.

- REQ-2.5: Upon receiving a declined payment status, the system shall prompt the user to try a different payment method and halt the reservation process.

5.3. Reception Operations

5.3.1 Description and Priority

This feature provides the necessary tools for the Receptionist to authenticate and manage daily guest arrivals and departures. Priority: Medium.

5.3.2 Stimulus/Response Sequences

Condition: The Receptionist submits valid login credentials.

Response: The system grants access to the Receptionist dashboard.

Condition: A Guest arrives, and the Receptionist initiates a check-in.

Response: The system updates the reservation and room status as officially occupied.

5.3.3 Functional Requirements

- REQ-3.1: Upon submitting credentials, the system shall execute login() to verify the employeeID and password against the database.
- REQ-3.2: Upon successful authentication, the system shall redirect the Receptionist to the active dashboard.
- REQ-3.3: Upon a guest's physical arrival, the Receptionist shall be able to execute manageCheckIn() to officially start the reservation period.
- REQ-3.4: Upon a guest's physical departure, the Receptionist shall be able to execute manageCheckOut() to conclude the reservation and trigger updateStatus(status: false) to free the room.

5.4. Administration Operations

5.4.1 Description and Priority

This feature outlines the exclusive tools available to the Admin user for overseeing hotel operations and employee access. Priority: Low (Operational).

5.4.2 Stimulus/Response Sequences

Condition: The Admin requests a monthly operational report.

Response: The system compiles database data and displays the report.

Condition: The Admin registers a new staff member.

Response: The system creates a new Staff profile and grants login access.

5.4.3 Functional Requirements

- REQ-4.1: Upon request, the Admin shall be able to execute generateReports() to retrieve aggregated data on room occupancy and generated revenue.
- REQ-4.2: The system shall allow the Admin exclusive rights to execute manageStaff() in order to add, edit, or revoke employeeID credentials.

6. Nonfunctional Requirements

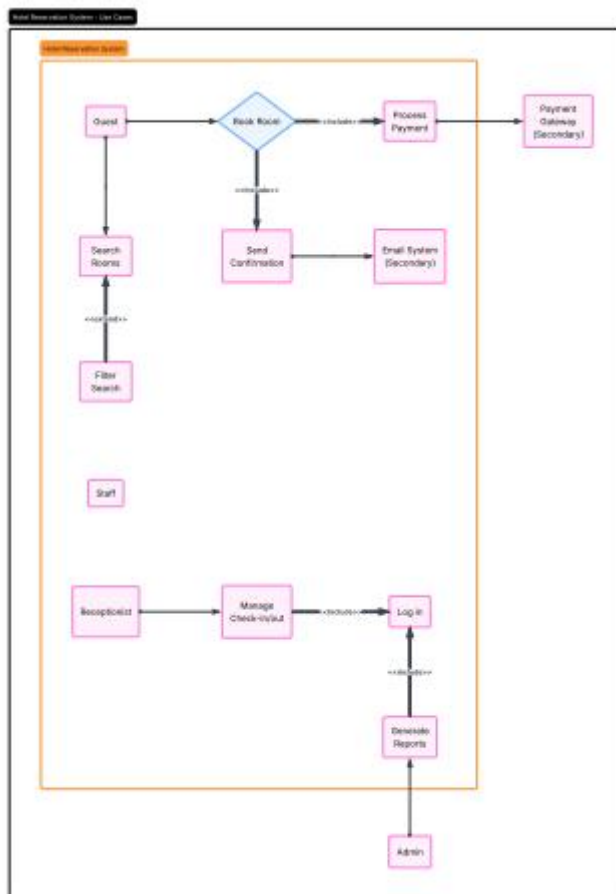
6.1. Performance

- NF-1.1: The system shall return room search results (getAvailableRooms()) in under 2 seconds under normal network conditions.
- NF-1.2: The web application shall be capable of handling up to 500 concurrent guest sessions without experiencing a degradation in response times exceeding 5 seconds.
- NF-1.3: Database queries related to updating the room occupancy status (isOccupied) shall be executed with immediate consistency to prevent double-booking anomalies.

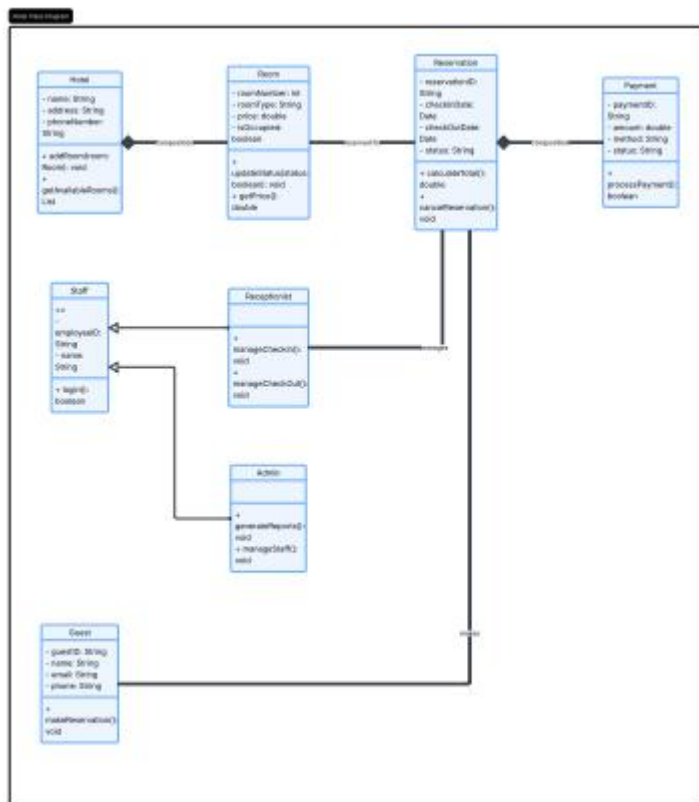
6.2. Security

- NF-2.1: All data transmitted between the client browser and the server (including personal guest data and staff login credentials) shall be encrypted using TLS 1.2 or higher (HTTPS).
- NF-2.2: The system shall not store raw credit card numbers in the local database; all financial transactions shall be handled via secure tokenization through the PCI-DSS compliant Payment Gateway.
- NF-2.3: All staff and admin passwords shall be salted and hashed in the database using industry-standard cryptographic algorithms (e.g., bcrypt or Argon2).
- NF-2.4: The system shall automatically terminate staff and admin login sessions (login() token expiration) after 30 minutes of inactivity to prevent unauthorized terminal access at the reception desk.

USE CASE DIAGRAM



CLASS DIAGRAM



ACTIVITY DIAGRAM

